Day 27 Notes: Kafka & Spark Streaming

1. Curriculum Topics

- **Topic:** Kafka & Spark Streaming
- **Concepts Covered:** Real-time data processing, Structured Streaming, Integration patterns.

1.1 Real-Time Data Processing

Real-time data processing involves ingesting and analyzing data as it is generated, minimizing latency between data arrival and insights. Apache Kafka serves as the durable message broker, while Apache Spark acts as the distributed computation engine.

1.2 Spark Structured Streaming

- **Micro-Batching:** By default, Spark groups incoming real-time data into small batches and processes them using the DataFrame API.
- **Unbounded Tables:** A data stream is treated as a table being continuously appended. Every new message in Kafka becomes a new row.
- **Exactly-Once Semantics:** Through checkpointing, Spark ensures each message is processed exactly once.

1.3 Integration Patterns

1. **Source:** A producer writes payloads to a Kafka topic.
2. **Ingestion:** Spark subscribes to the Kafka topic.
3. **Transformation:** Spark deserializes the binary message and applies transformations.
4. **Sink:** Spark writes the processed stream to a destination.

---

2. Problem Statement: Real-Time Fraud Detection

We will implement a real-time fraud detection pipeline. The architecture includes:

- **Producer:** Simulates transactions and pushes them to the `fraud-detection` Kafka topic.

- **Processing:** Spark reads transactions, filters anomalies (amount > 10,000), enriches the stream with user details from a CSV, and pushes alerts to the `fraud-notification` topic.
- **Consumer:** A Python script listens to `fraud-notification`. Users "log in" via their `userId` to receive real-time alerts.

3. Infrastructure Setup (docker-compose.yml)

Based on your specific Apache Kafka (KRaft mode) and Jupyter setup:

```
services:
  spark-jupyter:
    build: .
    container_name: spark-jupyter
    ports:
      - "8080:8080"
    volumes:
      - .:/workspace
    depends_on:
      - kafka

  kafka:
    image: apache/kafka:3.9.1
    container_name: kafka
    hostname: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      CLUSTER_ID: MkU3OEVBNTcwNTJENDM2Qk
```

4. Code Implementation

4.1 The Producer (producer.ipynb)

```
from kafka import KafkaProducer
import json
import time
import random

producer = KafkaProducer(
    bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

user_ids = [101, 102, 103, 104, 105]

print("Starting Transaction Producer...")
for i in range(100):
    tx_data = {
        "tx_id": i,
        "userId": random.choice(user_ids),
        "amount": random.uniform(10.0, 15000.0), # >10000 triggers fraud
        "timestamp": time.time()
    }
    producer.send('fraud-detection', value=tx_data)
    print(f"Sent: {tx_data}")
    time.sleep(2)
```

4.2 Spark Stream Processing (fraud_processing.ipynb)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col, struct, to_json
from pyspark.sql.types import StructType, StructField, IntegerType, DoubleType

# Initialize Spark with Kafka package
spark = SparkSession.builder \
    .appName("FraudDetection") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

# 1. Load Static User Data (CSV)
users_df = spark.read.csv("user_table.csv", header=True, inferSchema=True)

# 2. Read Streaming Data from Kafka
tx_schema = StructType([
    StructField("tx_id", IntegerType(), True),
    StructField("userId", IntegerType(), True),
    StructField("amount", DoubleType(), True),
    StructField("timestamp", DoubleType(), True)
])

kafka_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "fraud-detection") \
    .load()

# 3. Parse and Filter Data
parsed_stream = kafka_stream.select(from_json(col("value").cast("string"),
tx_schema).alias("tx")).select("tx.*")
fraud_stream = parsed_stream.filter(col("amount") > 10000.0)

# 4. Enrich Stream with User Details
enriched_fraud = fraud_stream.join(users_df, "userId")

# 5. Format for output Kafka topic
output_stream = enriched_fraud \
    .withColumn("value", to_json(struct("*")).cast("string")) \
    .select("value")

# 6. Write Stream to 'fraud-notification' Topic
query = output_stream.writeStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("topic", "fraud-notification") \
    .option("checkpointLocation", "/workspace/checkpoints") \
    .start()

query.awaitTermination()
```

4.3 User App & Consumer (user_app.py)

```
from kafka import KafkaConsumer
import json

def user_login_and_listen():
    print("=== Fraud Alert System ===")
    user_id_input = input("Enter your userId to login: ")

    try:
        user_id = int(user_id_input)
    except ValueError:
        print("Invalid ID. Exiting.")
        return

    print(f"Logged in as User {user_id}. Listening for alerts...")

    # Listen to kafka:9092
    consumer = KafkaConsumer(
        'fraud-notification',
        bootstrap_servers=['kafka:9092'],
        auto_offset_reset='latest',
        value_deserializer=lambda x: json.loads(x.decode('utf-8'))
    )

    for message in consumer:
        alert_data = message.value
        if alert_data.get('userId') == user_id:
            print("\n[CRITICAL ALERT]")
            print(f"Name: {alert_data.get('name')}")
            print(f"Tx ID: {alert_data.get('tx_id')}")
            print(f"Amount: ${alert_data.get('amount'):.2f}\n")

if __name__ == "__main__":
    user_login_and_listen()
```